

Computational Machine Learning (COSC2673)

Assignment 2: Machine Learning Classification and Model Comparison

Date due: 11th May 2026

Name: Vadhthanak Vibol

ID: s3951117

File: s3951117_report

1 Introduction

1.1 What is the problem?

Wildfires are a significant environmental and economic concern, causing damage to ecosystems, property, and most importantly, human life. The problem addressed in this assignment is to predict the level of fire intensity based on environmental and satellite-derived features such as brightness, temperature, humidity, wind speed, and precipitation.

This is formulated as a supervised machine learning classification problem, where the goal is to learn patterns from historical data and classify new observations into different levels of fire intensity.

1.2 Why the problem is important

Accurate predictions in fire intensity can significantly help authoritative prepare for a forest fire when it happens and can provide people with means to reduce wildfires moments before one gets any worse.

This could save many lives and the environment, causing less damage to both environments and economical concerns.

1.3 Machine Learning Task

By using classification problems where the input features (X) that includes different variables for instance: brightness, temperature, humidity, wind speed and precipitation, and comparing the target variable (y) which represents fire intensity.

Three classification models will be trained and compared to determine which model performs the best in predicting wildfire intensities most accurately.

2 Data

2.1 Data Description

The Data set contains the predictions of the intensity of fires or how strong the wildfires will be when it occurs by using different variables to help predict it.

2.2 Features - Before Data Cleaning

2.2.1 Categorical Features:

- acq_date (string)
- season (string)
- daynight (string)
- region (string)
- country (string)
- fire_type (string)
- satellite (string)
- instrument (string)
- confidence (string)

2.2.2 Numerical Features:

- latitude (float)
- longitude (float)
- acq_time (integer)
- year (integer)
- month (float)
- brightness_k (float)
- temp_max_c (float)
- wind_max_kmh (float)
- precip_mm (float)
- humidity_pct (float)

2.2.3 Target Variable:

- fire_intensity (string: Low / Moderate / High / Extreme)

2.2.4 Important Note

Certain features contains missing values (NaN). *month*, *brightness_k*, and *wind_max_kmh* have null values compared to other columns and all these missing values will be handled during the data cleaning stage before the machine learning models are to be trained.

2.3 Data Cleaning

Seeing that there is NaN values (month, brightness and wind speed) and many redundant values, a proper cleaning of the dataset is required for the models to be trained efficiently.

2.3.1 Date Time Values

Firstly looking at the date and time section of the table, it is observed that the months have several NaN values in it's columns that makes model training inconsistent and inaccurate. To combat this problem, we fortunately have another column that provides exactly what is needed to fill in the NaN values. The table called *acq_date* provides the year, month, and day giving us the month needed to fill in the missing values. To do this, *acq_date* will be converted to a datetime format and the month will be taken from *acq_date* to put in month values, eliminating the missing rows.

Messy data can still be encountered in this state if we were to look at the hour columns. Each row has many unique data where the hour is included with minutes making the data have necessary noise. Resolving this is to remove the minutes completely and keeping the hour digits. This makes it have only 24 unique values, cleaning the data and making the data more readable by machine learning models.

2.3.2 Median Values

In the case of brightness and wind speed there also exists NaN values that will hinder the progress and effectiveness of the machine learning models.

The method being used to remove these NaN values is median imputation which will be applied to both *brightness_k*, *wind_max_kmh* due to presence of outliers and skewed distributions

Unlike using the mean, median is more useful to be used with extreme values and is better for the typical observation. More complex methods such as interpolation or model-based imputation were not used, as they may introduce unrealistic patterns or unnecessary complexity at this stage of preprocessing.

2.3.3 Redundant Data

Final part of data cleaning is removing redundant data for the machine learning model to utilize a cleaner and more efficient table. Columns such as satellite and instrument both have unique data* making the information repetitive and unnecessary, therefore it is good practice to remove one of them from the table. In the case of this dataset, the instrument column will be removed in favor for satellite to be used.

*Using the code `(train.groupby('satellite')['instrument'].nunique())`

2.3.4 Data Types

Other important data to be cleaned is the *fire_intensity* column and it is required for it to be in the values of *int64* being:

- 0 for Low
- 1 for Moderate
- 2 for High
- 3 for Extreme

3 Exploratory Data Analysis (EDA)

3.1 Frequency

The first data to be analyzed is the frequency of each value's appearance in the *fire_intensity* column. Finding the data for this is extremely useful as we can start to see patterns initially by telling which is the most frequent occurring type of wild fire with the data being:

- Moderate: 1921
- High: 1340
- Low: 698
- Extreme: 381

This suggest most wildfires detected are Moderate while follwing up with high then low, and extreme being the least common.

Observing the data, it is also shown that the dataset is imbalanced with Moderate being the most frequent class and Extreme the least. The imbalance here may bias models into predicting the majority classes (Especially Moderate fire intensity). Therefore, macro F1-score will be used as the primary evaluation metric to ensure equal importance across all classes.

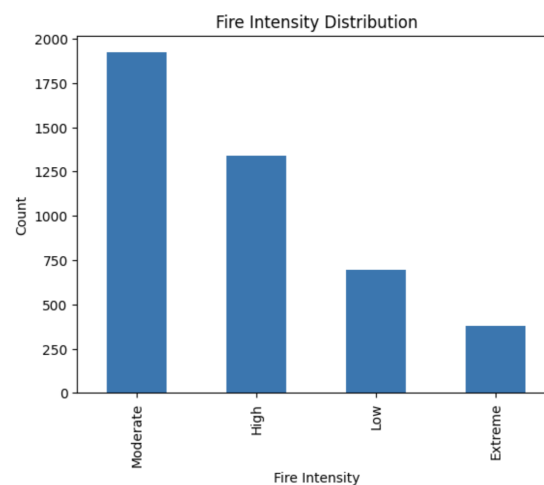


Figure 1: Frequency distribution of fire intensity levels

3.2 EDA Describe

By using the code `train.describe()`, it gives us very useful insights towards the data being gathered for the machine learning process.

3.3 Correlation Heatmap

Using a heatmap to analyze correlation between data is an essential way to confirm how each variable interacts with each other, but more importantly how it interacts with the target variable *fire_intensity*.

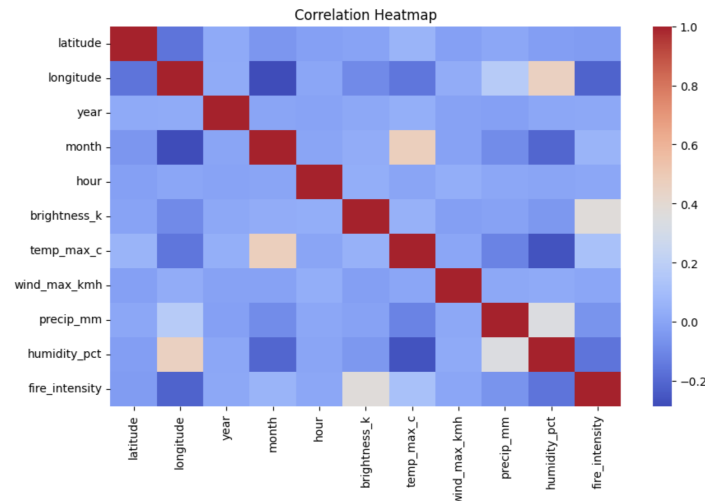


Figure 2: Correlation heatmap

Looking at figure 2 and focusing on the fire intensity row/column there are many interactions. Some of which are stronger than most for example:

- Brightness having the strongest correlation of them all,
- Temperature having a weak positive correlation with fire intensity.
- Humidity shows a negative relationship, suggesting higher humidity is associated with lower fire intensity.
- Precipitation showing a slight negative correlation with fire intensity.
- Wind speed does not show a significant relationship with fire intensity.

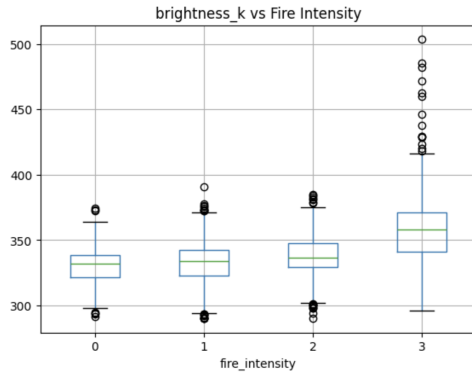
3.3.1 Variables and Fire intensity relationship

Here we will have a deep look in how the different variables relate to the main prediction variable fire intensity and how each of them effect the data.

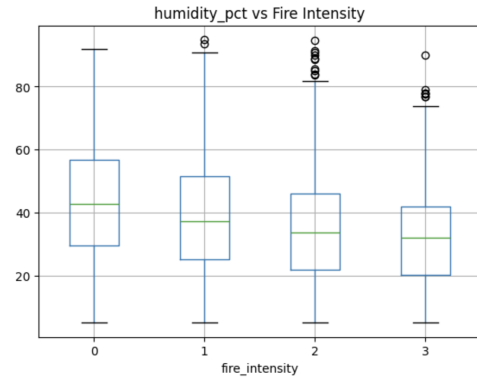
- Brightness (*brightness_k*): Brightness shows a strong relationship on fire intensity. As we can observe in Figure 3(a), it is shown that the higher the median for the brightness level, the higher intensity the wild fires with level 3 fire intensity having the highest median.
- Humidity (*humidity_pct*): As for humidity, the effect it has on fire intensity is the opposite of brightness as the lower the humidity mean the higher chance of a higher

level of wild fires most likely due to less moisture in the air. The graph shows a negative relationship between the two variables.

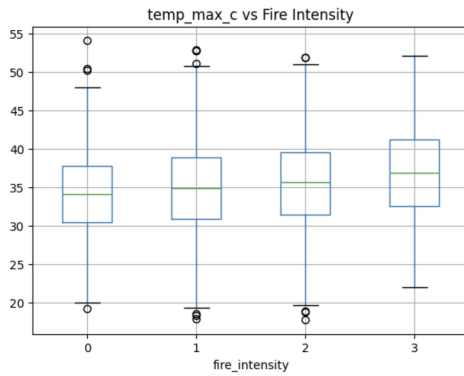
- Temperature (*temp_max_c*): Temperature slightly increases with fire intensity suggesting a weak positive relation ship. Considering that it is the air temperature, it seems like its the other way around where the wildfires are what is effecting the air temperatre instead.
- Wind speed (*wind_max_speed*): Wind speed shows no clear relationship with fire intensity as the median is all the same across the board. However considering how many outliers we have for the wind speed can suggest that the wild fires can be caused and spread by unusually high windspeeds.



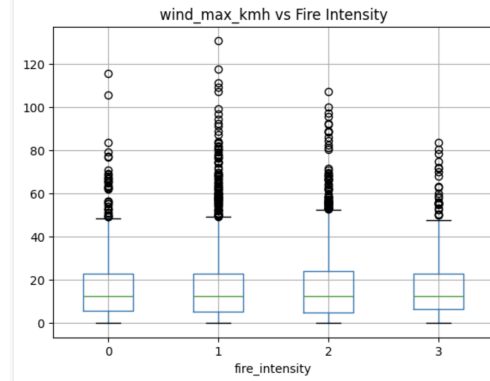
(a) Brightness vs Fire Intensity



(b) Humidity vs Fire Intensity



(c) Temperature vs Fire Intensity



(d) Wind vs Fire Intensity

Figure 3: Relationships between variables and fire intensity

4 Preprocessing

Data preprocessing is needed to convert raw, noisy, and unstructured data into clean, usable formats for analysis or machine learning.

4.1 Train split/validation

After the dataset was cleaned and split into training plus validation sets using a standard 80/20 split ratio with stratification applied to the target variable.

Stratified splitting makes sure that each wildfire intensity class maintains a similar distribution across both the training and validation datasets. This is important for balance between fire intensity classes.

The training set is used to train the machine learning models while the validation set is used to evaluate model performance on unseen data.

4.2 One-hot encoding

Machine learning models cannot directly process categorical string values such as season, satellite, confidence, and fire type as they are in different values such as int64, float, string. Therefore, one-hot encoding was applied to convert categorical variables into numerical binary columns, these binary columns help the models read each column and use the boolean to influence the data.

This allows the machine learning models to understand and process categorical information effectively.

4.3 Feature Scaling

Feature scaling was applied using `StandardScaler()` for Support Vector Machine (SVM) and Neural Network models.

Scaling ensures that all numerical features are placed on a similar scale, preventing variables with larger values from dominating the learning process.

Decision Trees on the other hand do not require feature scaling therefore the raw data was copied into a different variable called *x_train_raw* and *X_val_raw* which is used for both training and validation checks for the decision tree.

4.4 Evaluation Metric

Macro F1-score was selected as the primary evaluation metric due to the imbalanced nature of the dataset. Having a dataset that is highly skewed towards the moderate intensity level so F1-score will be used to find the best kind of model to use to predict the fire intensity for the test results.

Unlike accuracy, Macro F1-score gives equal importance to all wildfire intensity classes including minority classes such as Extreme fire intensity.

5 Model Training

Now onto the main section which is model training. This section will help determine the first steps in selecting the best model to predict the results for the test data to see how the fire intensity matches the data.

The Hyperparameters will be tuned at a later section as this section will have a look at the baseline performance of each model and figure how well each model performs without any tuning.

5.1 Reason for models

Decision Trees were selected because they are easy to use, simple to read, and able to handle both numerical and categorical-style processed features without needing feature scaling. Not only that, but they are also useful for identifying how feature-based decision rules affect wildfire intensity classification.

However, Decision Trees can easily overfit complex datasets if their depths are too deep. This makes them useful for analysing model complexity and generalisation behaviour through hyperparameter tuning such as `max_depth`.

Support Vector Machines (SVMs) were used due to being effective for classification problems involving high-dimensional feature spaces and can create strong decision boundaries between classes. The SVMs include different kernels that also allows the model to capture both linear and non-linear relationships within wildfire intensity data.

Even so, SVMs can become sensitive to hyperparameters such as kernel choice, C , and γ . They also require feature scaling and may struggle to perform when datasets contain overlapping or noisy class distributions.

Neural Networks were chosen because they are capable of learning complex non-linear relationships between environmental variables and wildfire intensity levels. This makes them best for the datasets where interactions between variables may not be straightforward.

Having said that, Neural Networks are much more computationally expensive and sensitive to hyperparameters such as learning rate and network structure. They may also overfit training data if not tuned correctly/carefully.

5.2 Decision Tree

The Decision Tree model was first trained using default parameters to establish a baseline performance.

To do this, a scikit-learn method called **DecisionTreeClassifier** was used. The model works by repeatedly splitting the dataset into smaller decision branches based on feature values for it to classify wildfire intensity levels. During training, the model learns patterns from the training dataset and predicts the most suitable fire intensity class for each wildfire event.

Metric	Score
Training F1	1.000
Validation F1	0.362

Table 1: Baseline Decision Tree performance

The untuned Decision Tree shows severe overfitting, with perfect training performance but significantly lower validation performance. This suggests that the model memorises the training data and struggles to generalise to unseen wildfire samples.

5.3 SVM

The Support Vector Machine (SVM) model was trained using scaled data.

To implement the SVM model, the scikit-learn method **SVC** was used. The model attempts to separate wildfire intensity classes by finding an optimal boundary between data points. A linear kernel was first used for the baseline model before testing with different kernels and C values during the hyperparameter tuning.

Different kernel functions including linear, radial basis function (RBF), and polynomial kernels were tested to determine the best-performing configuration. After selecting the best kernel, the hyperparameter C was tuned to further improve model performance.

Metric	Score
Training F1	0.330
Validation F1	0.311

Table 2: Baseline SVM performance

The SVM model shows relatively similar training and validation F1-scores, indicating less overfitting compared to the Decision Tree. However, the overall performance remains lower, suggesting weaker predictive capability on the wildfire dataset.

5.4 Neural Networks

The Neural Network model was implemented using a Multi-Layer Perceptron (MLP) classifier.

To implement the Neural Network model, the scikit-learn method **MLPClassifier** was used. The model consists of interconnected hidden layers which learn patterns from the wildfire dataset through iterative training. During training, the model adjusts internal weights to improve its ability to classify wildfire intensity levels.

The model was trained using scaled data and different learning rates were tested during hyperparameter tuning to determine the best-performing configuration.

Metric	Score
Training F1	0.902
Validation F1	0.342

Table 3: Baseline Neural Network performance

The baseline Neural Network shows signs of overfitting, with much higher training performance compared to validation performance. Despite this, the validation score remains higher than the baseline SVM model, indicating stronger learning capability.

6 Hyperparameter Tuning

Hyperparameter tuning was performed to improve model performance and to analyse how different parameter choices affected training and validation results. The main evaluation

metric used was Macro F1-score, as the dataset is imbalanced and Macro F1 gives equal importance to each fire intensity class.

6.1 Decision Tree Hyperparameter Tuning

For the Decision Tree model, the main hyperparameter tested was `max_depth`. This parameter controls the maximum depth of the tree and directly affects model complexity. A shallow tree may be too simple and underfit the dataset, while a very deep tree may memorise the training data and overfit.

The tested values were 3, 5, 7, 10, 15, 20, 25, 30, 35 and `None`. While the value `None` represents an unrestricted tree depth, meaning the model can continue splitting until it reaches pure leaves or no further useful split is possible, this is similar to the method we first used without hyperparameter tuning.

<code>max_depth</code>	Training F1	Validation F1
3	0.374	0.357
5	0.373	0.331
7	0.474	0.352
10	0.606	0.313
15	0.802	0.371
20	0.931	0.342
25	0.988	0.344
30	0.998	0.357
35	1.000	0.362
<code>None</code>	1.000	0.362

Table 4: Decision Tree hyperparameter tuning results

The results show that increasing tree depth generally increased training F1-score. This is due to deeper trees having to be able to create more specific decision rules and fit the training data more closely. However, the validation F1-score did not increase consistently with depth. For example, a `max_depth` = 10 produced a higher training score than shallower trees, but its validation score decreased which suggests that it has some overfitting. This and the fact that sometimes going even higher can decrease the validation score even lower or make it more unpredictable, so to get the best results we need to land on the optimal depth.

The best validation performance was achieved at `max_depth` = 15, with a validation Macro F1-score of 0.371. While this model still shows some overfitting, it still provides the best balance among the tested Decision Tree configurations. After a depth of 35 was used the results remained the same which may suggest that the max depth for training might be limited to 35. The unrestricted tree which is similar to `max_depth` = 35 achieved a perfect training F1-score of 1.000. However, its validation score was lower than the best tuned tree, confirming that allowing the tree to grow without restriction leads to memorisation rather than better generalisation.

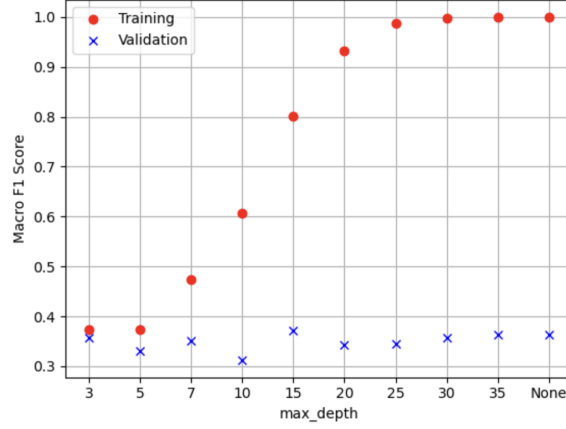


Figure 4: Graph of Tuned Decision Tree values

6.2 SVM Hyperparameter Tuning

For the Support Vector Machine model, the first step to determine which is the best tuning value to use is to find the best kernel first. The kernel will help determine how the SVM separates classes in the feature space. Four total kernels were tested: linear, radial basis function (RBF), polynomial, and sigmoid.

Kernel	Training F1	Validation F1
Linear	0.330	0.311
RBF	0.391	0.301
Polynomial	0.454	0.279
Sigmoid	0.290	0.329

Table 5: SVM kernel tuning results

Comparing the four results, it is evident that the Sigmoid Kernel achieved the highest validation Macro F1-score. Although the other three (Linear, RBF, Polynomial) kernels achieved higher training scores, their validation scores were lower, which suggests that these kernels did not generalise as well on this dataset. This suggests that the relationships in the processed feature space may be better handled by a Sigmoid kernel.

After selecting the sigmoid kernel, the regularisation parameter C was tuned. The C parameter controls the trade-off between allowing classification errors and fitting the training data more strictly. A smaller C value allows a wider margin with more tolerance for misclassification, while a larger C attempts to classify the training data more strictly.

Different gamma values were tested to observe their effect on validation Macro F1-score and overall model performance. They were tuned for the SVM model. The gamma values control how strongly individual data points impact the decision boundary. Smaller gamma values create smoother and more generalised boundaries, while larger gamma values produce more complex boundaries that may overfit the training data.

C Value	Training F1	Validation F1
0.01	0.153	0.153
0.1	0.201	0.195
1	0.290	0.329
10	0.272	0.354
100	0.288	0.375

Table 6: SVM C parameter tuning results

The best SVM validation result was achieved with $C = 100$, producing a validation Macro F1-score of 0.375. Increasing the C value generally improved validation performance, suggesting that a stronger decision boundary helped the model better classify wildfire intensity levels. Although the training F1-score did not increase substantially, the higher validation score indicates improved generalisation to unseen data.

After selecting the best C value, gamma tuning was performed for the sigmoid kernel. The best gamma result was achieved with $\text{gamma} = 0.01$, which produced a validation Macro F1-score of 0.390. Other gamma values such as 0.001, 0.1, and 1 resulted in lower validation performance. Therefore, the final selected SVM configuration was $\text{kernel} = \text{sigmoid}$, $C = 100$, and $\text{gamma} = 0.01$.

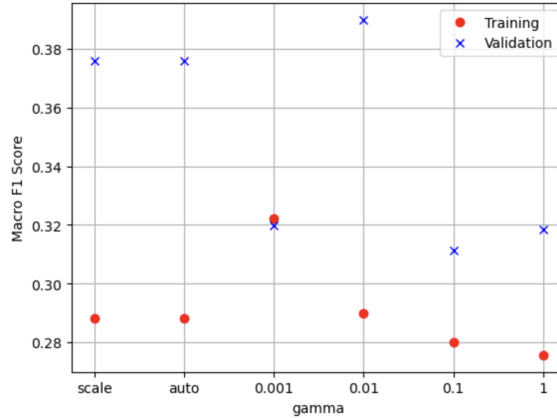


Figure 5: Graph of Tuned SVM values with gamma

6.3 Neural Network Hyperparameter Tuning

For the Neural Network model, the learning rate was tuned. The learning rate controls how large the model's weight updates are during training. If the learning rate is too small then the model may learn slowly. If it is too large, the model may update weights too aggressively and fail to generalise well.

The tested learning rates were 0.0001, 0.001, 0.01, 0.1 and 1. The model used one hidden layer with 64 neurons and a maximum of 500 iterations.

Learning Rate	Training F1	Validation F1
0.0001	0.561	0.392
0.001	0.902	0.342
0.01	0.954	0.358
0.1	0.635	0.397
1	0.373	0.326

Table 7: Neural Network learning rate tuning results

The results show that the smallest learning rate, 0.1, achieved the highest validation Macro F1-score of 0.397. Although it had a lower training F1-score than most of the learning rates, it generalised better to the validation set. This suggests that a learning rate that is in the middle ground has a more stable learning process and assisted the model to avoid excessive overfitting.

The learning rates 0.001 and 0.01 achieved much higher training F1-scores, but their validation scores were lower. This indicates that the model fitted the training data more but did not transfer this improvement to unseen validation hyper data. Therefore, `learning_rate_init = 0.1` was selected as the best Neural Network hyperparameter.

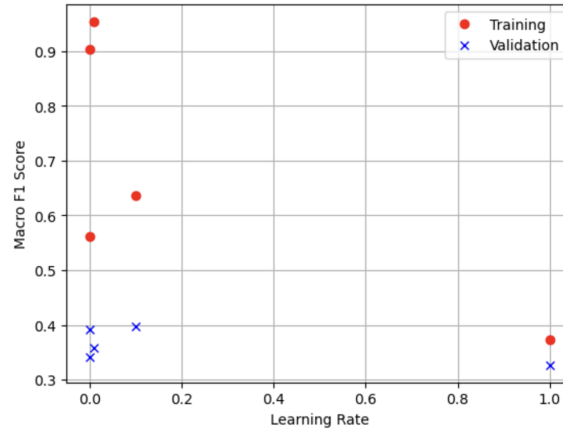


Figure 6: Graph of Tuned Neural Network values

A convergence warning appeared during Neural Network training, which means the optimiser has reached the maximum number of iterations before fully converging. However, since the model still produced valid validation results and the comparison was conducted consistently across learning rates, the results remain useful for model selection. Increasing the maximum number of iterations could be explored in future work, but 500 iterations was kept to maintain a consistent and manageable experiment.

7 Model Comparison and Predictions

Comparing all the results between the three classification models, it is observed that Neural Network had the best performance out of all with a balanced fit for the training F1 score and the highest validation F1 score. With a training score of 0.635 giving it a balanced fit compared to SVM's under fitting score for the training and Decision Tree's

overfitting score of a perfect 1.000. As for validation scores, while each of the models have almost similar scores, the Neural Network has the highest of them all being the chosen model for the model training with a learning rate of 0.1 being the tuned hyperparameter.

The results show that more complex models do not always perform better. Although the Decision Tree achieved perfect training performance, it generalised very poorly due to overfitting, especially when it achieved a perfect F1 training score of 1.000 . In comparison, the Neural Network achieved better validation performance despite having lower training performance, demonstrating that balanced generalisation is more important than memorising the training dataset.

	Model	Best Hyperparameter	Train F1	Validation F1
2	Neural Network	learning_rate = 0.1	0.635667	0.397285
1	SVM	kernel = sigmoid, C = 100, gamma = 0.01	0.289790	0.389876
0	Decision Tree	max_depth = 15	1.000000	0.371319

Figure 7: Model Comparison

7.1 Test cleaning

The test dataset was cleaned using the same preprocessing steps applied the same way to the training dataset for consistency between the two datasets. Date features were converted into month and hour values, unused columns were removed, missing values were filled using median imputation, and categorical variables were one-hot encoded.

Finally, the test dataset columns were aligned with the training dataset to ensure compatibility before prediction.

7.2 Predictions

After choosing the final model which is the Neural Network, it was retrained using the full cleaned training dataset. Feature scaling was applied by using `StandardScaler` to ensure consistent feature magnitudes between the training and test datasets.

The trained model was then used to predict wildfire intensity values for the test dataset. And lastly, predictions were exported into a CSV submission file following the required assignment format called `s3951117_predictions.csv`.

8 Conclusion

In conclusion, the report focuses on predicting the intensity of wildfires that are happening all across the globe by factoring different variables given to it with the dataset to help prepare firefighters, civilians, governments with combating and lessening the fire damage overall. These variables include brightness, temperature, humidity, seasons, precipitation and much more. While certain variables may play a larger factor than others in predicting the wildfires, it is also important to not completely rule out the lesser variables as they can still sway the outcome of the intensity.

While the dataset given was a great start, it remains flawed as there contains NaN values and redundant data which causes unnecessary data to be obtained. Data cleaning and reprocessing was utilised to tidy up the data by using methods such as median imputation, date-time conversion and so on. Which then leads to the analysis of the data including how each variable relate to each other. By doing so, it's possible to observe how much each data relate to one another. These observations can help with information such as how well each variable relates to the main variable and how swayed or noisy the data is.

Three models in total were trained: Decision tree, Support Vector Machines and Neural Networks. Each model was trained and tuned to optimise them and see which of the models performed best. While the Decision Tree achieved very high training performance, it showed strong signs of overfitting due to the large gap between training and validation scores. However, the SVM model produced much more stable results but achieved lower overall validation performance. Among all tested models, the Neural Network achieved the highest validation Macro F1-score and demonstrated the strongest overall predictive capability on unseen wildfire data.

Despite achieving reasonable predictive performance, the dataset still contains several limitations. Different wildfire intensity classes appeared more frequently than others that cause class imbalance which affects the how the mode's generalisation across the classes. Another factor to consider is that wildfire behaviour is highly complex and is only predictable to a limited extent and may be influenced by other different environmental factors that are not included within the dataset.

Future improvements could include collecting larger and more balanced datasets or introducing additional environmental variables such as vegetation density or drought measurements, and experimenting with more advanced machine learning methods. These improvements might improve predictions making them more accurate in future wildfire intensity prediction systems.

All in all, the results that this report gives is that machine learning models are highly capable of identifying meaningful wildfire intensity data and patterns from environmental and satellite data. Though there are still many limitations and many chances to improve on the results, the models developed throughout the report shows that machine learning models are useful tools in helping predicting fire intensities in wildfires which can reduce risk and have a better management to dealing with the wildfires.

9 References

- Scikit-learn Developers. *Scikit-learn Machine Learning Library*. Available at: <https://scikit-learn.org/>
- Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- RMIT University COSC2673/2793 Machine Learning course materials and workshop notes.